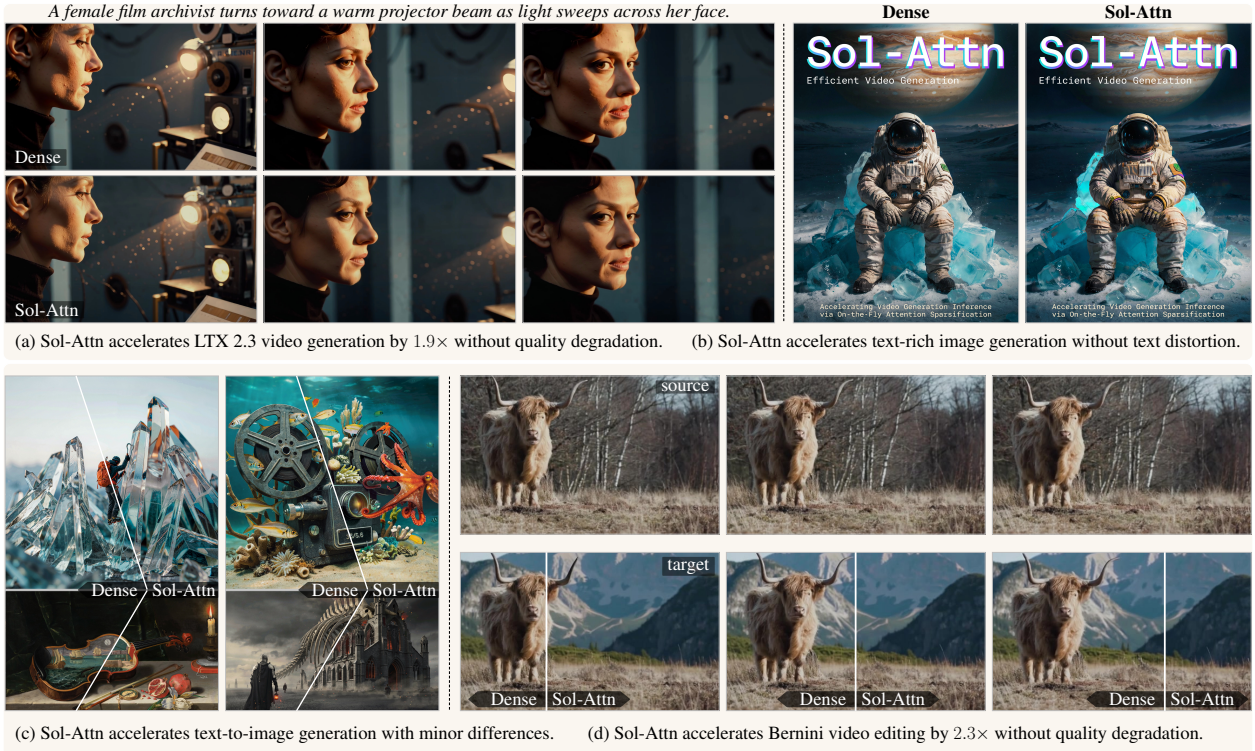
Sol-Attn: Accelerating Video Generation Inference via On-the-Fly Attention Sparsification

Haopeng Li, Yitong Li, Junsong Chen, Tian Ye, Haozhe Liu, Jincheng Yu, Duomin Wang, Ruihua Zhang
Zeke Xie, Enze Xie, Song Han
NVIDIA

Diffusion transformers are essential for high-fidelity video generation, but long token sequences make attention a dominant inference bottleneck. Training-free dynamic sparse attention alleviates this bottleneck by computing only selected key-value blocks, yet existing methods struggle to sparsify attention both efficiently and accurately for two reasons: (1) Rigid, unpredictable, and costly routing: selecting a fixed fraction of top-ranked blocks by proxy score imposes fixed budgets, whereas retaining blocks to reach a target cumulative proxy probability mass yields dynamic but potentially imbalanced budgets; both incur non-negligible overhead from computing and materializing proxy scores. (2) Lossy keep-or-drop sparsification: unselected blocks are discarded entirely, degrading accuracy under aggressive sparsity. These limitations motivate cheaper dynamic-budget routing while limiting accuracy degradation. In this paper, we introduce training-free ***Sol-Attn*** (*Sparsifying online attention*) that unifies dynamic routing, sparse computation, and approximation correction in a single online-softmax pass, achieving a better accuracy–efficiency trade-off in sparse attention. The core of Sol-Attn is **on-the-fly block thresholding** with **proxy-score reuse**, which selects critical blocks by comparing block proxy scores against a threshold during online softmax. This design enables dynamic yet controllable block budgets without materializing the proxy map, while directly reusing the proxy scores of unselected blocks to approximate their contribution. Experiments across image and video generation tasks show that Sol-Attn advances the quality–efficiency frontier of training-free sparse attention, delivering $2.1\times$ and $2.3\times$ end-to-end speedups for video generation and editing, respectively, while preserving visual quality. Furthermore, for video generation, integrating Sol-Attn into Sol-Engine with complementary acceleration techniques delivers up to a $5\times$ end-to-end speedup.

Links: GitHub Code | Project Page



(a) Sol-Attn accelerates LTX 2.3 video generation by $1.9\times$ without quality degradation. (b) Sol-Attn accelerates text-rich image generation without text distortion.

(c) Sol-Attn accelerates text-to-image generation with minor differences. (d) Sol-Attn accelerates Bernini video editing by $2.3\times$ without quality degradation.

Figure 1 | Overview of generated samples comparing dense attention and Sol-Attn. Across diverse generation tasks, Sol-Attn substantially accelerates generation without noticeable quality degradation.

arXiv:2607.24027v1 [cs.CV] 27 Jul 2026

© 2026 NVIDIA. All rights reserved.

1. Introduction

Diffusion transformers (DiTs) [1] have become the foundation of video generation [2, 3, 4, 5, 6]. However, the pursuit of higher-resolution and longer-duration generation results in increasingly long token sequences, making the quadratic complexity of self-attention [7] a dominant inference bottleneck. Training-free dynamic sparse attention is an attractive remedy because it can accelerate pretrained models without changing weights [8, 9, 10, 11]. Yet current block-sparse methods still rely heavily on offline routing from compressed attention-score proxies and keep-or-drop sparsification, which expose two sharp limitations. **(L1) Block routing is rigid, hard to control, and costly.** For each query block, top-k selects the k key-value blocks with the highest proxy scores, thus enforcing a uniform budget across query blocks; top-p retains blocks until their cumulative probability mass reaches p, risking severe budget imbalance when attention patterns are diffuse or sink-dominated. Both strategies materialize the proxy score map and routing indices in HBM before sparse attention begins, incurring non-negligible memory traffic at long sequence lengths. **(L2) Keep-or-drop sparsification is lossy.** Unselected blocks are discarded entirely, even when they carry non-negligible attention mass, degrading accuracy under aggressive sparsity. These limitations point to a tension between routing efficiency and sparse-attention accuracy: *Can we achieve cheaper dynamic-budget routing while limiting accuracy degradation?*

We propose Sol-Attn, a training-free sparse attention that fuses threshold-based dynamic sparsification into online softmax and reuses proxy scores to approximate unselected blocks, all within a single kernel, thereby achieving a better accuracy–efficiency trade-off. This design is realized through three steps:

Query-dependent thresholding. We observe that attention scores in pretrained video models are often near-symmetric and normal-like, suggesting that block selection can be performed by threshold-based filtering. Based on this insight, we propose a query-dependent threshold estimated from lightweight score statistics such as mean and standard deviation. This threshold selects blocks under a dynamic yet controllable budget, without materializing the full proxy map.

On-the-fly sparsification. During online softmax, each query block scans the pooled-key sequence tile by tile to produce proxy scores. These scores are compared on chip with the query-dependent threshold, and blocks above the threshold are immediately dispatched to a nested sparse attention loop over the original key-value tiles. Since each score is consumed as it is generated, no full proxy score map or routing indices need to be stored in HBM.

Proxy-score reuse. Instead of dropping proxy scores after routing, Sol-Attn keeps them in registers during online softmax and reuses the below-threshold scores to approximate unselected blocks. This recovers part of the contribution from dropped blocks and reduces approximation error relative to dense attention. The exact and approximate branches share the same online-softmax state, unifying routing, sparse computation, and correction in one streaming operator.

Experiments across image generation, video generation, editing, and refinement show that Sol-Attn advances the quality–efficiency frontier of training-free sparse attention while preserving generation quality. It delivers $2.1\times$–$3.0\times$ end-to-end speedups across video tasks and reaches $5.1\times$ when integrated with complementary optimizations in Sol-Engine.

2. Preliminaries

Attention and Online Softmax. Given query, key, and value matrices $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{L\times d}$, where L is the sequence length and d is the feature dimension, softmax attention [7] computes the output $\boldsymbol{O} \in \mathbb{R}^{L\times d}$ (omitting the scale factor):

$$\boldsymbol{O} = \mathrm{Softmax}\left(\boldsymbol{Q}\boldsymbol{K}^\top\right)\boldsymbol{V}. \tag{1}$$

FlashAttention [12] introduces online softmax to avoid materializing the full attention matrix $\mathrm{Softmax}(\boldsymbol{Q}\boldsymbol{K}^\top) \in \mathbb{R}^{L\times L}$ in HBM via tiled on-chip computation, thereby reducing I/O overhead and eliminating the $\mathcal{O}(L^2)$ memory footprint. However, it still evaluates all query-key interactions and therefore keeps the quadratic compute cost.

Block-Sparse Attention. To reduce the $\mathcal{O}(L^2)$ computational complexity, sparse attention [8, 9, 10, 11, 13, 14, 15] computes only a subset of critical key-value pairs for each query. Mathematically, it can be formulated as:

$$\boldsymbol{O} = \mathrm{Softmax}\left(\boldsymbol{Q}\boldsymbol{K}^\top + \boldsymbol{M}\right)\boldsymbol{V}, \tag{2}$$

where $\boldsymbol{M} \in \{0, -\infty\}^{L\times L}$ is the attention mask, with $-\infty$ entries ignored by the softmax. To align sparse attention with hardware-friendly tiled online softmax, the sparse pattern is organized at block granularity, allowing masked key-value tiles to be skipped entirely and translating sparsity into compute savings.

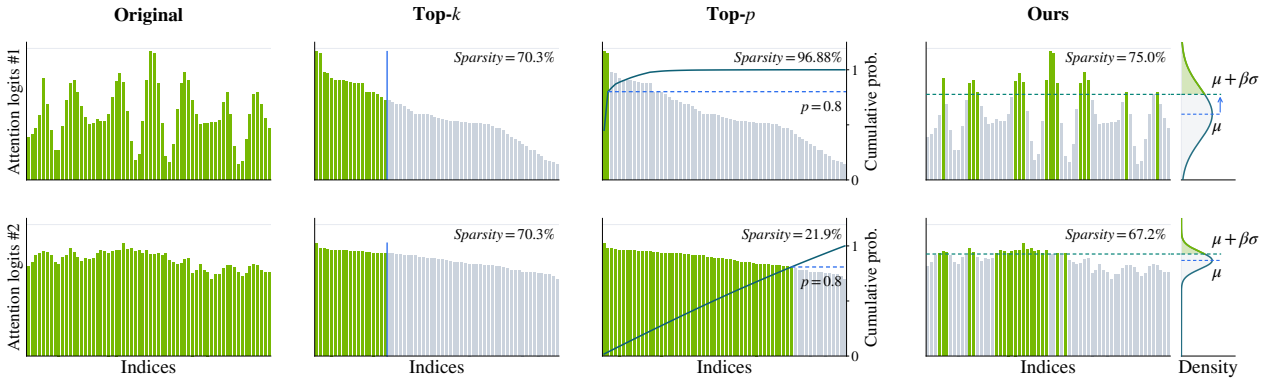


Figure 2 | **Comparison of routing strategies.** Bars represent pre-softmax block-proxy logits, with selected blocks highlighted in green. Top-k selects the highest-scoring blocks under a fixed budget. Top-p accumulates the softmax probabilities to a target p, yielding a dynamic budget that is sensitive to the probability distribution and thus hard to balance. Sol-Attn instead derives a threshold from the mean μ and standard deviation σ of the block-proxy logits, with β setting the standardized offset, yielding a dynamic yet controllable budget without materializing the block-proxy map.

To dynamically construct $\boldsymbol{M}$ under this block structure, standard methods typically rely on block-wise proxy attention scores. Let the sequence be partitioned into N blocks with block size B, and denote the i-th query block as $\boldsymbol{Q}_i \in \mathbb{R}^{B\times d}$ and the j-th key-value block as $\boldsymbol{K}_j, \boldsymbol{V}_j \in \mathbb{R}^{B\times d}$. The block scores are proxied by:

$$\widehat{s}_{ij} = \bar{\boldsymbol{Q}}_i \bar{\boldsymbol{K}}_j^{\top}, \qquad \widehat{\boldsymbol{p}}_i = \mathrm{Softmax}(\widehat{\boldsymbol{s}}_i), \tag{3}$$

where $\bar{\boldsymbol{Q}}_i := \mathrm{Mean}(\boldsymbol{Q}_i)$, $\bar{\boldsymbol{K}}_j := \mathrm{Mean}(\boldsymbol{K}_j) \in \mathbb{R}^{1\times d}$ denote means along the token dimension; $\widehat{\boldsymbol{s}}_i = [\widehat{s}_{i1}, \ldots, \widehat{s}_{iN}]$ collects the proxy scores between query block i and all key blocks. Standard routing applies top-k to $\widehat{\boldsymbol{s}}_i$ or top-p to $\widehat{\boldsymbol{p}}_i$ to select key-value blocks, which are then marked as active in the block-sparse mask $\boldsymbol{M}$. As discussed in Sec. 1 and illustrated in Figure 2, top-k enforces a rigid fixed budget, whereas top-p produces a dynamic but hard-to-control budget; both additionally traverse the complete proxy row before sparse computation, incurring non-negligible routing overhead.

3. Methodology

To enable dynamic sparse attention to adaptively allocate its compute budget across varying softmax-score distributions without costly routing, we propose Sol-Attn, a training-free sparse attention method for video diffusion transformers that unifies proxy routing, sparse computation, and approximate correction within a single online-softmax pass, thereby reducing routing overhead while improving approximation accuracy. We first derive query-dependent thresholds, then integrate routing into tile-wise online softmax, and finally reuse the tile-wise routing score for approximate correction.

3.1. Query-Dependent Thresholding

Structured Attention Logits. Across models, we aggregate the row-wise pre-softmax block-proxy logits $\hat{s}_{ij}$ in Eq. (3) over denoising steps, layers, attention heads, and query blocks. The resulting distribution is consistently near-Gaussian within each model, as shown in Figure 3. This aggregate structure provides a natural routing coordinate: for a Gaussian variable, a cutoff β standard deviations above the mean corresponds to a unique upper-tail density. A shared β can therefore control the overall sparsity level of a model, while each row maps this common standardized cutoff back to its own raw-logit scale through its row-wise mean and standard deviation rather than sharing a fixed raw-score cutoff.

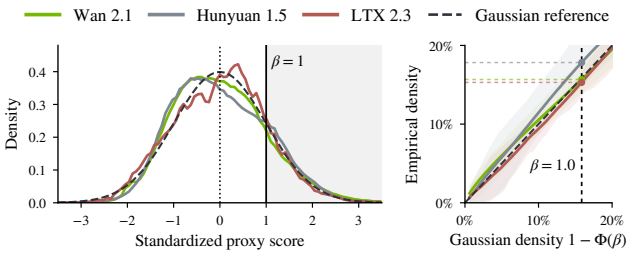


Figure 3 | **Gaussian-calibrated density control.** Standardized block-proxy logits are near-Gaussian (left); empirical densities match the predicted tail at cutoff β (right).

Threshold-Based Routing. Motivated by this observation, we propose query-dependent threshold routing. For query block i, let μ_i and σ_i denote the mean and standard deviation of its proxy row $\widehat{\boldsymbol{s}}_i$. Given a shared standardized cutoff $\beta > 0$, we define its routing threshold τ_i and the set of selected key-block indices $\mathcal{S}_i$ as

$$\tau_i = \mu_i + \beta\sigma_i, \qquad \mathcal{S}_i = \{j : \widehat{s}_{ij} > \tau_i\}. \tag{4}$$

Let Φ denote the standard Gaussian CDF. Under the Gaussian reference, β corresponds to the selected density $\rho_{\mathrm{G}}(\beta) := 1 - \Phi(\beta)$, while μ_i and σ_i map the shared standardized cutoff to the raw-score scale of query i. Figure 3 shows that the empirical mean densities remain close to $\rho_{\mathrm{G}}(\beta)$ across the evaluated models, while the shaded bands span the 10th–90th percentiles of the per-row densities. Thus, β calibrates the model-level mean density, whereas $|\mathcal{S}_i|$ remains query-dependent rather than fixed as in top-k. Top-p also permits variable counts, but p controls cumulative mass rather than block density: depending on the concentration of the softmax-score distribution, the same p may retain between one and $\lceil pN \rceil$ blocks. Threshold routing therefore combines a dynamic budget with an empirically calibrated density scale.

Efficient Threshold Computation. The remaining question is how to efficiently obtain τ_i without materializing the full proxy-score row $\widehat{\boldsymbol{s}}_i$. Since $\widehat{\boldsymbol{s}}_i$ is obtained by linearly projecting the pooled keys with $\bar{\boldsymbol{Q}}_i$, its mean and variance can be computed directly from the first and second moments of the pooled-keys (see Appendix B for the derivation):

$$\mu_i = \bar{\boldsymbol{Q}}_i \left(\frac{1}{N} \sum\nolimits_{j=1}^{N} \bar{\boldsymbol{K}}_j \right)^{\top}, \qquad \sigma_i^2 = \bar{\boldsymbol{Q}}_i \left(\frac{1}{N} \sum\nolimits_{j=1}^{N} \bar{\boldsymbol{K}}_j^{\top} \bar{\boldsymbol{K}}_j \right) \bar{\boldsymbol{Q}}_i^{\top} - \mu_i^2. \tag{5}$$

All row-wise means μ_i and variances σ_i^2, and hence the query-dependent thresholds τ_i, can therefore be computed in $\mathcal{O}(Ld + Nd^2)$ time using $\mathcal{O}(d^2)$ auxiliary storage, without materializing the $N \times N$ proxy score map.

3.2. Sol-Attn: Unified On-the-Fly Sparsification

Chunk-wise On-the-Fly Thresholding. Once the threshold τ_i for the i-th query block is available, each key-value block can be selected as soon as its proxy score is produced, without materializing the full proxy-score row. This property allows block selection to proceed directly within online softmax. We regard the block-wise pooled keys $\{\bar{\boldsymbol{K}}_j\}_{j=1}^{N}$ as a new key sequence of length N and partition it into T chunks of size C. For chunk t, let $\bar{\boldsymbol{K}}^{(t)} = [\bar{\boldsymbol{K}}_{tC+1}; \ldots; \bar{\boldsymbol{K}}_{(t+1)C}] \in \mathbb{R}^{C \times d}$. Each block $\boldsymbol{Q}_i$ scans these chunks sequentially to produce chunk-wise proxy scores:

$$\widehat{\boldsymbol{s}}_i^{(t)} = \bar{\boldsymbol{Q}}_i (\bar{\boldsymbol{K}}^{(t)})^{\top} \in \mathbb{R}^{1 \times C}. \tag{6}$$

Applying Eq. (4) to each chunk yields

$$\mathcal{S}_i^{(t)} = \left\{ tC + r \mid r \in \{1, \ldots, C\}, [\widehat{\boldsymbol{s}}_i^{(t)}]_r > \tau_i \right\}. \tag{7}$$

Since $\mathcal{S}_i = \bigcup_{t=0}^{T-1} \mathcal{S}_i^{(t)}$, chunk-wise streaming thresholding recovers the same block set as full-row thresholding. The selected blocks in each $\mathcal{S}_i^{(t)}$ are immediately consumed by sparse attention, naturally yielding a *nested-loop structure*:

- **Outer loop** densely scans the pooled-key sequence chunk by chunk to determine the chunk-wise selected set.
- **Inner loop** sparsely traverses the key-value blocks selected within the current chunk to perform attention computation.

This nested-loop structure enables on-the-fly attention sparsification that is mathematically equivalent to offline global thresholding, without materializing the full proxy-score map or routing indices as intermediate tensors.

Proxy-Score Reuse as Approximate Correction. Blocks whose proxy scores fall below the threshold would otherwise be discarded by sparse attention. To retain their contributions at low cost, we approximate their block-wise exponential score matrices. Specifically, we apply the Taylor expansion around the row-wise mean score within each key block:

$$\exp(\boldsymbol{Q}_i \boldsymbol{K}_j^{\top}) = \exp(\boldsymbol{Q}_i \bar{\boldsymbol{K}}_j^{\top}) \odot \left[\mathbf{1} + \boldsymbol{Q}_i (\boldsymbol{K}_j - \bar{\boldsymbol{K}}_j)^{\top} + \mathcal{O}\Big(\big(\boldsymbol{Q}_i (\boldsymbol{K}_j - \bar{\boldsymbol{K}}_j)^{\top} \big)^{\odot 2} \Big) \right], \tag{8}$$

where $\odot$ denotes element-wise multiplication and the $B \times 1$ center is broadcast along the key dimension. Retaining only the zeroth-order term approximates the entire block-wise exponential score matrix using its pooled key, thereby preserving approximate contributions from skipped blocks to both the softmax numerator and denominator.

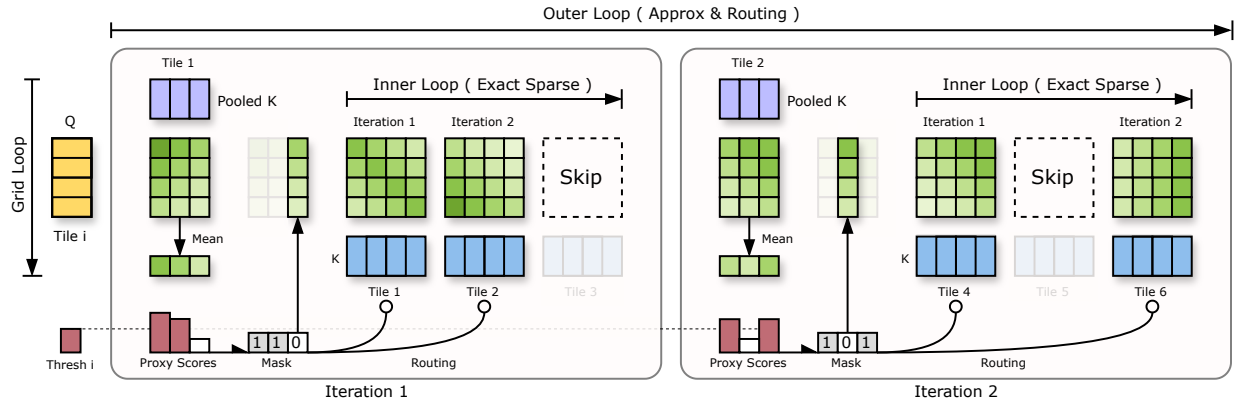


Figure 4 | **Sol-Attn execution pipeline**. At each outer-loop iteration, a query block multiplies a pooled-key tile (each pooled key is pooled from an original key tile) to produce token-to-block scores; averaging each column over query tokens yields the block-proxy scores. The selected blocks are dispatched to the exact inner loop, while the token-to-block scores are reused for approximation, with the selected columns masked to avoid double counting.

For each block j, let $\widehat{\boldsymbol{V}}_j \in \mathbb{R}^{1\times d}$ denote the sum of $\boldsymbol{V}_j$ along the token dimension, and let $\mathcal{U}_i := \{1,\dots,N\} \setminus \mathcal{S}_i$ denote the set of unselected blocks. Replacing the exponential scores of blocks in $\mathcal{U}_i$ with their zeroth-order approximations, while retaining exact computation for blocks in $\mathcal{S}_i$, yields the combined softmax denominator and numerator:

$$\boldsymbol{D}_i := \underbrace{\sum\nolimits_{j\in\mathcal{U}_i} B\cdot\exp\left(\boldsymbol{Q}_i\bar{\boldsymbol{K}}_j^\top\right)}_{\text{Approx.}} + \underbrace{\sum\nolimits_{j\in\mathcal{S}_i}\mathrm{RowSum}\left(\exp\left(\boldsymbol{Q}_i\boldsymbol{K}_j^\top\right)\right)}_{\text{Exact}} \in \mathbb{R}^{B\times 1}. \tag{9}$$

$$\boldsymbol{N}_i := \underbrace{\sum\nolimits_{j\in\mathcal{U}_i}\exp\left(\boldsymbol{Q}_i\bar{\boldsymbol{K}}_j^\top\right)\widehat{\boldsymbol{V}}_j}_{\text{Approx.}} + \underbrace{\sum\nolimits_{j\in\mathcal{S}_i}\exp\left(\boldsymbol{Q}_i\boldsymbol{K}_j^\top\right)\boldsymbol{V}_j}_{\text{Exact}} \in \mathbb{R}^{B\times d}. \tag{10}$$

Normalizing each row of $\boldsymbol{N}_i$ by the corresponding entry of $\boldsymbol{D}_i$ yields the output block $\boldsymbol{O}_i \in \mathbb{R}^{B\times d}$.

In practice, the per-block matrix–vector multiplication (GEMV) in the approximate terms of Eqs. (9) and (10) can be batched over a chunk of C pooled keys and evaluated as an efficient matrix–matrix multiplication (GEMM), producing a $B \times C$ token-to-block score tile. This tiled computation also exposes its direct computational overlap with routing: averaging each column over query tokens exactly recovers the chunk-wise routing scores in Eq. (6):

$$\widetilde{\boldsymbol{S}}_i^{(t)} := \boldsymbol{Q}_i(\bar{\boldsymbol{K}}^{(t)})^\top, \qquad \mathrm{Mean}\left(\widetilde{\boldsymbol{S}}_i^{(t)}\right) = \bar{\boldsymbol{Q}}_i(\bar{\boldsymbol{K}}^{(t)})^\top = \widehat{\boldsymbol{s}}_i^{(t)}. \tag{11}$$

As illustrated in Figure 4, block-wise approximation and chunk-wise routing are naturally unified in the outer loop. For each pooled-key chunk, the column means of the token-to-block score tile determine block selection; unselected columns provide approximate correction, while selected blocks are dispatched to the exact inner loop. Reusing the same score tile in this way fuses routing, sparse computation, and approximate correction within a single online-softmax pass.

Hardware-Aligned Implementation. We implement Sol-Attn using the nested-loop kernel shown in Figure 4 and summarized in Algorithm 1. Here, `KC[j]` and `VC[j]` correspond respectively to the pooled key $\bar{\boldsymbol{K}}_j$ and summed value $\widehat{\boldsymbol{V}}_j$ used by the approximate terms in Eqs. (9) and (10).

Each kernel program stages $\boldsymbol{Q}_i$ and τ_i in shared memory and maintains a single online-softmax state in registers. The outer loop streams chunks of `KC` and `VC`, while the inner loop loads the original $\langle \boldsymbol{K}_j, \boldsymbol{V}_j\rangle$ blocks only for routed indices. Both the approximate and exact paths update the same online-softmax statistics. Since routing is derived from the score tile already computed for approximation, its overhead is largely absorbed into the approximate online-softmax pipeline, without materializing proxy scores or routing indices in HBM.

Algorithm 1: Sol-Attn Forward Pass

```
def sol_attn(Q, K, V, O, KC, VC, tau, N, B, C):
    for i in range(N):
        Q_i = Q[i*B:(i+1)*B]
        acc, l_i, m_i = 0, 0, -inf
        # outer loop: routing and approximation
        for j in range(0, N, C):
            s_ij = QK_GEMM(Q_i, KC[j:j+C])
            mask = s_ij.mean(axis=0) > tau[i]
            s_ap = where(mask, -inf, s_ij)
            p_ap = Softmax(s_ap, acc, l_i, m_i)
            acc += PV_GEMM(p_ap, VC[j:j+C])
            # inner loop: exact sparse attention
            for t in nonzero(mask):
                s_ex = QK_GEMM(Q_i, K[(j+t)*B:(j+t+1)*B])
                p_ex = Softmax(s_ex, acc, l_i, m_i)
                acc += PV_GEMM(p_ex, V[(j+t)*B:(j+t+1)*B])
        O[i*B:(i+1)*B] = acc / l_i[:, None]
    return O
```

4. Experiments

4.1. Experimental Setup

Models. We evaluate text-to-video generation with Wan2.1-14B [2], HunyuanVideo-13B [3], and LTX 2.3-22B [4, 5], spanning increasingly long spatiotemporal sequences. Video-to-video evaluation covers editing with Bernini-14B [16] and refinement with SANA-WM Refiner [17]. We also evaluate native 2K text-to-image generation with Ideogram 4 [18] to assess the effectiveness of Sol-Attn at moderate sequence lengths.

Metrics. We measure task quality using VBench [19] for text-to-video generation, pose accuracy for SANA-WM refinement, Bernini-Bench [16] for Bernini editing, and Qwen-Image-Bench [20] for high-resolution text-to-image generation. Across tasks, PSNR, SSIM, and LPIPS quantify fidelity to dense attention outputs. Because theoretical FLOPs do not necessarily reflect practical efficiency, we report end-to-end wall-clock speedup relative to dense attention.

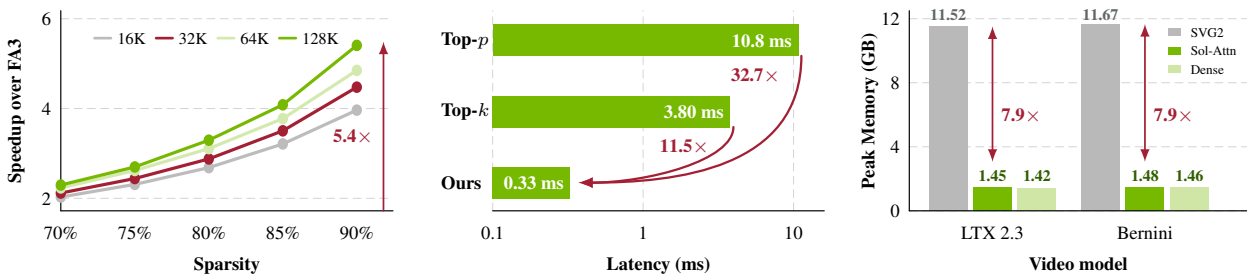
Baselines. We use FlashAttention-3 (FA3) [21] as the dense attention baseline and compare with recent state-of-the-art training-free sparse attention methods: XAttention (XAttn) [10], Sparse-VideoGen2 (SVG2) [8], and Piecewise Sparse Attention (PISA) [11]. For XAttn and SVG2, we adapt their officially recommended configurations to models not covered by the original releases, such as LTX, and match sparsity across methods for fair comparison.

Implementation Details. We implement Sol-Attn with custom GPU kernels and profile on NVIDIA H100 GPUs. Exact attention uses fixed 64×64 physical blocks. Since the routing and approximation chunk size C affects efficiency but not the mathematical output, we do not treat it as an ablation variable. Following prior work, for Wan2.1, HunyuanVideo, Bernini, and Ideogram 4, we use dense attention during the first 20% of denoising steps and in the first layer as a warm-up. For the two-stage LTX pipeline, stage-1 uses an 8-step LoRA with dense attention throughout, whereas stage-2 uses Sol-Attn for all steps without warm-up. The SANA-WM refiner likewise uses Sol-Attn throughout.

4.2. Kernel Efficiency Evaluation

Kernel Efficiency Evaluation. Figure 5a reports the kernel-level speedup of Sol-Attn over FlashAttention-3 across different sequence lengths and sparsity levels. The speedup increases consistently with both sequence length and sparsity, reaching $5.41\times$ at 128K tokens and 90% sparsity. This consistent scaling trend demonstrates that the efficiency advantage of Sol-Attn persists throughout the 16K–128K sequence-length range.

Efficient On-the-Fly Sparsification. Figures 5b and 5c isolate the two system costs of sparsification: selection latency and intermediate memory. Top-k and top-p materialize and globally traverse the proxy map, with top-p further requiring cumulative selection over sorted scores. Sol-Attn instead consumes each score on chip as it is produced, making routing $11.5\times$ and $32.7\times$ faster, respectively. Tile-local routing state also keeps Sol-Attn near dense attention in processor memory, whereas SVG2's routing structures and permutation buffers require about $8\times$ more memory. The consistent separation on LTX 2.3 and Bernini attributes both gains to on-the-fly sparsification rather than model-specific integration.



(a) Speedup vs. sparsity. (b) Routing latency (log scale). (c) Attention-processor memory.

Figure 5 | Kernel efficiency profiled on an NVIDIA H100 GPU. (a) Speedup over FA3 increases with sparsity and sequence length. (b) Threshold routing is faster than top-k and top-p. (c) Incremental peak memory of the attention processor. Sol-Attn remains near Dense, while SVG2 requires about $8\times$ more memory on both workloads.

Table 1 | Quantitative comparison of text-to-video quality and efficiency. LTX 2.3 uses a coarse-to-fine two-stage pipeline, with Sol-Attn applied only to stage-2; outside (inside) values denote end-to-end (stage-2) speedups.

Method	VBench↑									Similarity			Efficiency	
	SC	BC	TF	MS	AQ	IQ	DD	OC	AVG	PSNR↑	SSIM↑	LPIPS↓	Sparsity	Speedup↑
***Wan 2.1** (14B, Text-to-Video, 720p, 81 Frames)*														
FA3	95.59	96.75	98.75	97.86	63.17	68.88	61.11	25.08	75.90	–	–	–	–	1.00×
XAttn	93.71	95.43	97.94	97.12	60.56	66.12	62.50	25.37	74.84	18.62	0.5544	0.3226	85.60%	1.69×
SVG2	94.85	96.05	98.59	97.68	62.26	67.40	59.72	25.19	75.22	20.71	0.6543	0.2498	83.66%	1.85×
PISA	95.60	96.95	98.75	97.98	63.72	68.91	61.11	25.19	76.03	20.28	0.6775	0.2127	85.00%	1.86×
Sol-Attn	95.54	97.02	98.76	98.01	63.35	68.64	62.50	25.26	76.13	20.59	0.6825	0.2086	85.12%	2.02×
***HunyuanVideo** (13B, Text-to-Video, 720p, 129 Frames)*														
FA3	93.87	96.60	98.97	99.11	61.54	67.87	72.22	26.28	77.06	–	–	–	–	1.00×
XAttn	93.76	95.57	99.05	98.76	60.10	66.38	51.39	26.16	73.90	22.90	0.6792	0.2511	85.60%	1.61×
SVG2	92.22	95.63	99.01	98.95	58.01	61.81	63.89	26.08	74.45	25.64	0.7853	0.2354	83.15%	2.01×
PISA	93.68	96.37	98.95	99.08	61.69	67.71	72.22	26.49	77.02	25.58	0.8164	0.1671	85.00%	1.88×
Sol-Attn	93.62	96.49	98.95	99.05	61.52	67.55	70.83	26.50	76.81	25.21	0.8048	0.1777	85.80%	2.12×
***LTX 2.3** (22B, Text-to-Video, 1080p, 361 Frames → 721 Frames)*														
FA3	90.78	95.53	99.31	99.33	62.59	69.89	56.94	22.38	74.59	–	–	–	–	1.0×
XAttn	90.04	95.17	98.86	98.99	62.56	70.21	58.33	22.55	74.59	21.77	0.7621	0.2175	90.17%	1.6 (1.9)×
SVG2	90.70	95.46	99.28	99.06	59.74	64.86	44.44	22.28	71.98	21.94	0.7806	0.2874	89.35%	1.7 (2.1)×
PISA	90.89	95.64	99.37	99.39	62.76	70.10	56.94	22.27	74.67	25.26	0.8630	0.1288	90.00%	1.8 (2.3)×
Sol-Attn	90.85	95.67	99.36	99.39	62.92	70.16	56.94	22.24	74.69	25.69	0.8704	0.1197	89.83%	1.9 (2.4)×

Table 2 | Comparison of one-minute video refinement with the SANA-WM refiner across different attention methods.

Method	Pose Acc.↓			Similarity			Efficiency	
	R.Err.	T.Err.	CMC	PSNR↑	SSIM↑	LPIPS↓	Sparsity	Speedup
FA3	7.133	1.167	1.216	–	–	–	–	1.00×
XAttn	10.52	1.331	1.415	16.80	0.466	0.411	85.30%	2.17×
SVG2	9.109	1.413	1.477	16.89	0.473	0.413	85.08%	2.08×
PISA	8.822	1.237	1.299	17.65	0.506	0.351	84.96%	2.35×
Sol-Attn	8.781	1.233	1.296	17.72	0.507	0.343	85.11%	3.04×

Table 3 | Comparison of video-to-video editing with the Bernini-14B across different attention methods.

Method	Bernini-Bench↑				Similarity			Efficiency	
	IF	VC	GQ	OS	PSNR↑	SSIM↑	LPIPS↓	Sparsity	Speedup
FA3	3.32	3.80	3.70	3.41	–	–	–	–	1.00×
XAttn	3.46	3.65	3.80	3.46	27.42	0.864	0.079	84.91%	1.64×
SVG2	3.41	3.62	3.62	3.36	27.28	0.863	0.074	81.15%	2.04×
PISA	3.36	3.75	3.75	3.55	29.88	0.904	0.052	85.00%	2.17×
Sol-Attn	3.50	3.80	3.80	3.50	30.18	0.910	0.046	85.00%	2.34×

4.3. Visual Generation Evaluation

Text-to-Video Generation. We evaluate text-to-video generation across three models, spanning short 720p videos to long 1080p videos. Table 1 compares Sol-Attn with recent training-free sparse attention methods in terms of VBench quality and similarity to dense-attention outputs, while speedup is measured end-to-end relative to dense attention with the FA3 backend. At matched sparsity, Sol-Attn achieves state-of-the-art efficiency across all models, reaching up to $2.12\times$ end-to-end speedup while overall outperforming other methods in VBench quality and dense-reference similarity.

Video-to-Video Generation. We further evaluate Sol-Attn on video-to-video generation, a more demanding regime with substantially longer sequences. We consider two 720p settings: refining a one-minute draft video with SANA-WM Refiner and editing videos with Bernini. Tables 2 and 3 report task-specific quality, dense-reference similarity, and end-to-end speedup at matched sparsity. On SANA-WM, Sol-Attn achieves the best pose accuracy and dense-reference similarity among sparse methods while delivering a $3.04\times$ speedup. On Bernini-Bench, it achieves the strongest aggregate generation quality and dense-reference similarity among sparse methods, together with the highest speedup.

Text-to-Image Generation. Beyond long-sequence video generation, we evaluate Sol-Attn under moderate sequence lengths in native 2K text-to-image generation. Table 4 shows that, on Ideogram 4, Sol-Attn outperforms other sparse attention methods in both Qwen-Image-Bench scores and dense-reference similarity while achieving a $1.56\times$ speedup.

Video Generation on Consumer GPUs. We further evaluate Sol-Attn on the consumer-grade NVIDIA RTX 5090 GPU. Figure 6 reports the kernel speedup over FlashAttention-4 [22], while Table 5 compares different sparse attention methods with Wan2.1-1.3B for 480p video generation. Sol-Attn still achieves the best overall quality and efficiency.

Table 4 | Comparison of attention methods on Ideogram 4 for 2K-resolution text-to-image generation. BSA denotes standard block sparse attention.

Method	Qwen-Image-Bench↑						Similarity			Efficiency	
	Qual.	Aesth.	Align.	Fidel.	Creat.	AVG	PSNR	SSIM	LPIPS	Sparsity	Speedup
FA3	56.10	60.88	59.74	56.40	64.95	59.24	–	–	–	–	1.00×
BSA	50.04	52.28	52.63	51.40	53.08	51.96	15.99	0.565	0.427	90.00%	1.54×
PISA	52.78	55.56	55.09	53.07	55.85	54.51	20.12	0.738	0.239	90.00%	1.47×
Sol-Attn	52.77	56.12	56.61	53.33	57.71	55.31	21.26	0.764	0.210	90.16%	1.56×

Table 5 | Comparison of Wan2.1-1.3B 480p video generation on the RTX 5090.

Method	Similarity			Efficiency	
	PSNR	SSIM	LPIPS	Sparsity	Speedup
FA4	–	–	–	–	1.00×
XAttn	15.16	0.41	0.554	86.09%	1.36×
SVG2	24.08	0.77	0.195	86.00%	1.41×
Sol-Attn	24.70	0.794	0.162	84.90%	1.71×

4.4. Integration with Sol-Engine

Since Sol-Attn sparsifies attention itself, it is compatible with complementary training-free acceleration techniques, including diffusion-step caching [23, 24, 25] and kernel fusion. We therefore integrate it into the Sol-Engine [26], a full-stack inference framework for composing such techniques, and evaluate the resulting system on NVIDIA B200 GPUs. Figure 7 shows that the combined system achieves end-to-end speedups of 3.48× on Wan2.1-14B and 5.08× on HunyuanVideo, demonstrating that Sol-Attn is a practical drop-in component for optimized inference engines.

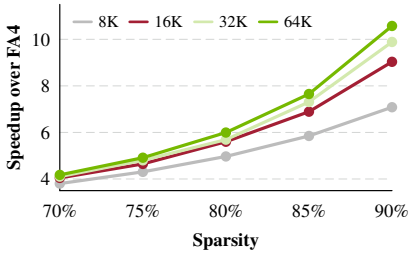


Figure 6 | Sparsity versus speedup over FA4 on an NVIDIA RTX 5090.

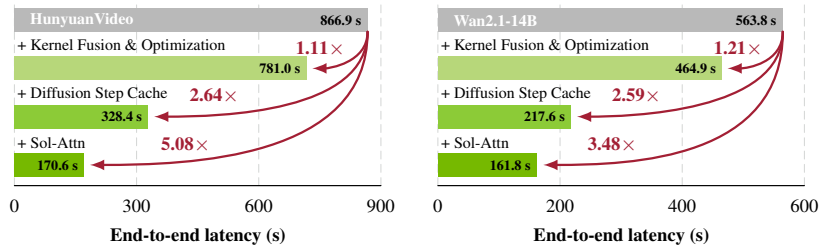


Figure 7 | Integrating Sol-Attn into Sol-Engine. End-to-end latency as acceleration techniques are progressively incorporated.

4.5. Ablation Study

Effect of Query-Dependent Thresholding. Figure 8 compares the per-query-block density distributions of different routing strategies at the same mean density of 15%. Top-k fixes every query block at the target density by construction, whereas Top-p produces broadly varying densities. In contrast, our method remains tightly concentrated across models, demonstrating that it enables dynamic block budgets while maintaining stable density control.

Effect of Approximate Correction. Figure 9 evaluates approximate correction by comparing standard sparse attention (exact-only) and Sol-Attn (exact-or-approx) under identical selected block indices as sparsity increases, measuring relative ℓ_2 error and cosine similarity against dense attention outputs. Standard sparse attention accumulates rapidly growing error, whereas approximate correction consistently reduces relative ℓ_2 error and preserves higher cosine similarity, with a widening advantage at higher sparsity. This confirms that retaining an approximate contribution from unselected blocks substantially improves the accuracy of sparse attention.

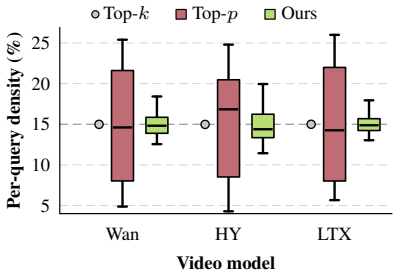


Figure 8 | Routing-density distributions across different models.

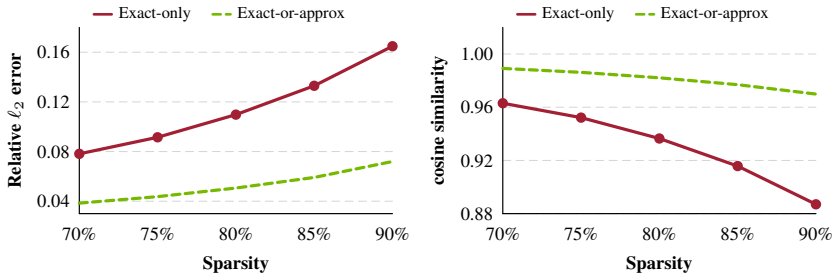
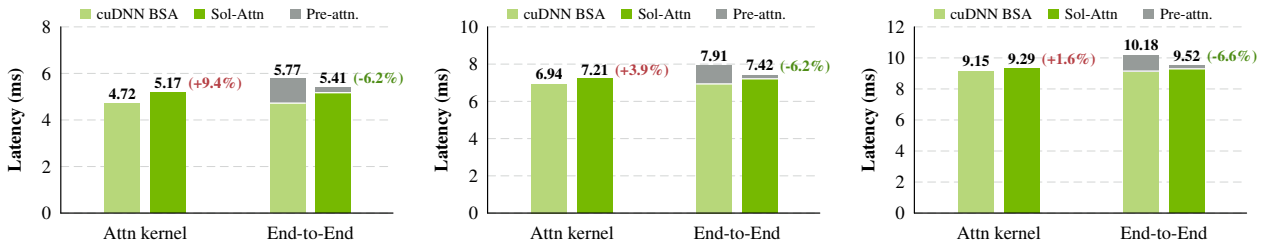


Figure 9 | Approximate-correction ablation at a sequence length of 32K across sparsity levels: relative ℓ_2 error (left) and mean row cosine similarity (right).



(a) 90% sparsity. (b) 85% sparsity. (c) 80% sparsity.

Figure 10 | Latency breakdown at a sequence length of 32K. Attn Kernel measures the attention operator alone, whereas End-to-End measures the full QKV-to-output path. The approximate correction in Sol-Attn introduces only minor kernel overhead. At the end-to-end level, Sol-Attn is faster because it computes only lightweight thresholds and hides routing overhead within the online softmax pipeline, whereas BSA materializes routing indices externally.

Efficient Unified Execution Pipeline. To break down the latency of our unified kernel, we compare Sol-Attn against cuDNN BSA, the state-of-the-art block-sparse attention implementation on H100 GPUs, using identical block indices at 10%, 15%, and 20% exact density. Since Sol-Attn fuses routing and approximate correction into the online-softmax pipeline of sparse attention, its kernel performs more work than BSA. Nevertheless, Figure 10 shows only 9.4%, 3.9%, and 1.6% kernel-level overhead, respectively, confirming that approximate correction is lightweight. The comparison reverses end to end once BSA's separate routing stage is included: Sol-Attn is 6.2%, 6.2%, and 6.6% faster because only threshold computation remains outside its attention kernel, while the routing work is largely hidden by other operations in the online-softmax pipeline. Unified execution thus makes on-the-fly routing nearly free in the end-to-end pipeline.

5. Related Work

Block Sparse Attention. Block sparse attention reduces quadratic attention cost by computing only selected key-value blocks. Early methods reduce this cost through structured layouts [27, 28, 29] or content-based grouping [30, 31]. More recent work learns block selection or combines compression, selection, and local attention within a native sparse architecture [32, 33, 34]. In video generation, existing methods exploit the spatial–temporal redundancy of DiTs in two broad ways. One line of work constructs structured sparse masks from recurring spatial or temporal attention patterns [35, 36, 37, 38]; another derives content-adaptive sparse patterns at runtime from lightweight block-level proxy scores [8, 39, 40, 41], achieving more flexible block selection.

Routing Strategies. Dynamic sparse attention hinges on selecting informative key-value blocks efficiently. The most direct strategies apply top-k or top-p to block-proxy scores. SpargeAttn augments top-k selection with a confidence threshold and then applies an online-softmax-aware threshold to skip negligible updates [9]. SpargeAttn2 [42] takes the union of top-k and top-p masks to avoid their complementary failure modes [42], whereas Twilight [43] first forms a conservative top-k candidate set and then prunes it with top-p [43]. Other methods redesign the importance measure or filtering primitive: XAttention scores blocks by antidiagonal sums [10], while VecAttention retains scores within a fixed margin of the running row maximum and fuses the comparison into tiled score computation [44]. Related systems reduce the search cost through query-aware page selection [45], hierarchical pruning [46], context-dependent sparse patterns [47], or learned semantic hashing [48]. Sol-Attn instead computes a query-dependent threshold for each proxy row and applies it to scores as they are generated, enabling routing to proceed online within the online-softmax pass.

Sparse Attention with Correction. Correction-based methods approximate contributions omitted by the exact sparse branch. PISA applies block-wise Taylor expansion to unselected blocks and combines exact and approximate terms within the same softmax numerator and denominator [11]. SVG-EAR uses centroid-based linear compensation with error-aware routing [49], and BA-Att introduces covariance-compensated block approximation [50]. SLA assigns critical weights to sparse attention and marginal weights to linear attention [51]; SLA2 adds learned routing, weighted sparse–linear composition, and quantization-aware training [52]. Existing corrections either follow a sparse pattern constructed in a separate routing stage or require model fine-tuning. Sol-Attn instead derives routing and correction from the same token-to-block score tile in a single online-softmax pass.

6. Conclusion

We presented Sol-Attn, a training-free sparse attention that combines threshold routing and approximate correction in a single online-softmax pass without materializing a proxy map. More broadly, our study explores a new form of sparse attention in which routing is no longer a separate prerequisite and unselected blocks are approximated rather than discarded. This unified streaming design substantially narrows the accuracy gap between sparse and dense attention with little additional overhead, yielding a favorable quality–efficiency trade-off across diverse visual-generation tasks.

Limitations and Future Work. Despite these encouraging results, Sol-Attn remains limited in both its current implementation and application scope. The B200 kernel does not yet fully exploit Blackwell's performance potential and currently supports only forward inference, while our evaluation is confined to bidirectional diffusion-based visual generation and does not cover autoregressive video generation. Future work will address these limitations through further kernel optimization, extension to autoregressive models, and a trainable implementation with backward support.

A. Additional Experimental Details

Generation Configurations. Wan2.1-14B and HunyuanVideo-13B use 50 denoising steps, with dense attention for the first 10 steps and Sol-Attn thereafter. Bernini-14B follows the same policy over 40 steps with an eight-step dense warm-up. Wan2.1 and Bernini keep cross-attention dense, while HunyuanVideo's MMDiT treats text key-value tokens as sinks and retains them in the exact branch. Ideogram 4 uses eight dense steps in its 48-step schedule and likewise retains text key-value tokens in the exact branch. For LTX 2.3, the eight-step first stage remains dense, and Sol-Attn is applied throughout the remaining three steps while cross-attention stays dense. The three-step SANA-WM refiner uses Sol-Attn throughout and retains dense cross-attention.

B. Derivations and Error Analysis

Threshold variance derivation. The exact variance in Eq. (5) follows directly from the variance of the proxy row. Recall from Eq. (3) that $\widehat{s}_{ij} = \bar{\boldsymbol{Q}}_i \bar{\boldsymbol{K}}_j^\top$. The corresponding row mean is $\mu_i = \frac{1}{N}\sum_{j=1}^{N} \widehat{s}_{ij}$, its variance expands as

$$\begin{aligned}
\sigma_i^2 &= \frac{1}{N}\sum\nolimits_{j=1}^{N} (\widehat{s}_{ij} - \mu_i)^2 \\
&= \frac{1}{N}\sum\nolimits_{j=1}^{N} \left(\widehat{s}_{ij}^2 - 2\mu_i \widehat{s}_{ij} + \mu_i^2\right) \\
&= \frac{1}{N}\sum\nolimits_{j=1}^{N} \widehat{s}_{ij}^2 - 2\mu_i \left(\frac{1}{N}\sum\nolimits_{j=1}^{N} \widehat{s}_{ij}\right) + \mu_i^2 \\
&= \frac{1}{N}\sum\nolimits_{j=1}^{N} \widehat{s}_{ij}^2 - \mu_i^2.
\end{aligned} \tag{12}$$

Each squared proxy score can in turn be expressed as

$$\widehat{s}_{ij}^2 = \left(\bar{\boldsymbol{Q}}_i \bar{\boldsymbol{K}}_j^\top\right)\left(\bar{\boldsymbol{Q}}_i \bar{\boldsymbol{K}}_j^\top\right)^\top = \bar{\boldsymbol{Q}}_i \bar{\boldsymbol{K}}_j^\top \bar{\boldsymbol{K}}_j \bar{\boldsymbol{Q}}_i^\top. \tag{13}$$

Since $\widehat{s}_{ij}$ is scalar, the second factor can be replaced by its transpose to expose the pooled-key second moment. Substituting Eq. (13) into Eq. (12) and collecting the key-dependent terms then gives

$$\sigma_i^2 = \bar{\boldsymbol{Q}}_i \left(\frac{1}{N}\sum\nolimits_{j=1}^{N} \bar{\boldsymbol{K}}_j^\top \bar{\boldsymbol{K}}_j\right) \bar{\boldsymbol{Q}}_i^\top - \mu_i^2. \tag{14}$$

This recovers Eq. (5).

Diagonal threshold estimator. We provide implementations of both the exact estimator in Eq. (5) and a more efficient approximation that retains only the diagonal of the pooled-key covariance. Let $\boldsymbol{\mu}_K := \mathbb{E}[\bar{\boldsymbol{K}}]$ and retain only the diagonal $\boldsymbol{v}_K := \mathrm{diag}(\mathrm{Cov}[\bar{\boldsymbol{K}}])$. The resulting estimator is

$$\boldsymbol{v}_K = \frac{1}{N}\sum\nolimits_{j=1}^{N} \bar{\boldsymbol{K}}_j \odot \bar{\boldsymbol{K}}_j - \boldsymbol{\mu}_K \odot \boldsymbol{\mu}_K, \qquad \sigma_{i,\mathrm{diag}}^2 = (\bar{\boldsymbol{Q}}_i \odot \bar{\boldsymbol{Q}}_i)\boldsymbol{v}_K^\top. \tag{15}$$

It replaces the dense $d \times d$ covariance with element-wise second moments and an $O(d)$ projection per query block.

Approximation error. The zeroth-order rule also gives a direct error characterization. For one query row, write $a = \boldsymbol{q}\bar{\boldsymbol{K}}_j^\top$, $\delta_u = \boldsymbol{q}(\boldsymbol{k}_{j,u} - \bar{\boldsymbol{K}}_j)^\top$, and $\eta = \max_u |\delta_u|$. Let $\mathcal{D}_j, \widetilde{\mathcal{D}}_j$ and $\mathcal{N}_j, \widetilde{\mathcal{N}}_j$ denote the exact and zeroth-order block denominator and numerator contributions for this row. Since $\sum_u \delta_u = 0$, Taylor's theorem yields

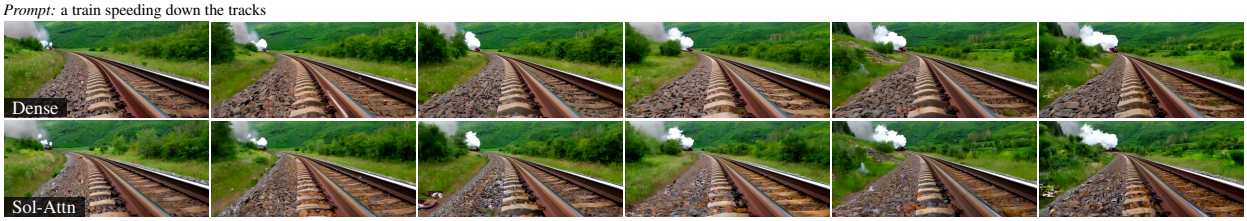
$$0 \le \mathcal{D}_j - \widetilde{\mathcal{D}}_j \le \frac{1}{2}\exp(a+\eta)\sum_{u=1}^{B} \delta_u^2, \qquad \|\mathcal{N}_j - \widetilde{\mathcal{N}}_j\|_2 \le \exp(a+\eta)\sum_{u=1}^{B} |\delta_u| \|\boldsymbol{v}_{j,u}\|_2. \tag{16}$$

Thus, the denominator error is second order in the centered score spread, whereas the numerator also depends on its alignment with the values. These bounds explain the accuracy on smooth tail blocks and the residual error on heterogeneous ones. The derivation assumes dense, noncausal keys within each block, as in the evaluated diffusion models; row-dependent semantic or causal masks instead require centering and multiplicity over valid keys.

C. Additional Qualitative Results

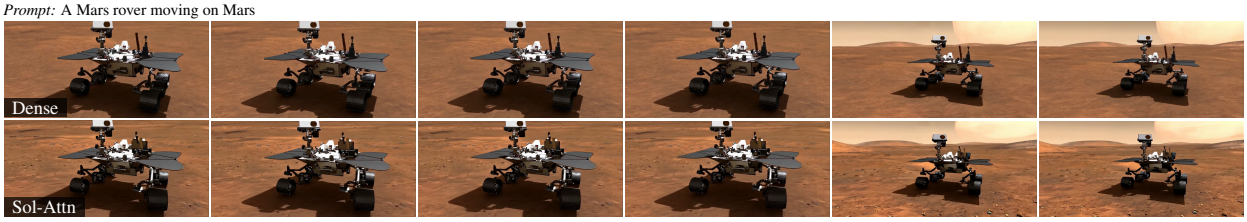
Prompt: a motorcycle cruising along a coastal highway

Prompt: a train speeding down the tracks



(a) Wan 2.1

Prompt: A Mars rover moving on Mars



Prompt: A steam train moving on a mountainside

(b) HunyuanVideo

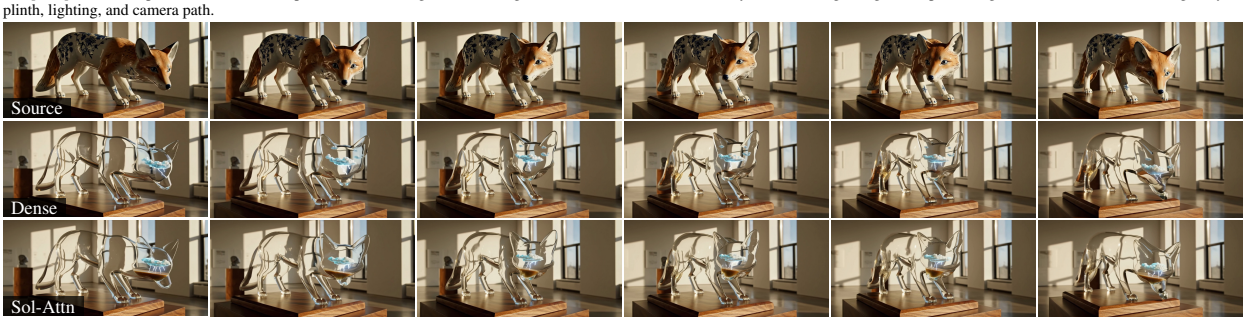
Prompt: INT. QUIET APARTMENT BY TALL WINDOW - LATE AFTERNOON. A bright cinematic 85mm close-up frames a beautiful woman in a cream sweater beside white curtains and wooden blinds. Her complete face fills the frame, with soft skin texture, clear eyes, and a calm natural expression; the background is a warm blurred room with plants and books, not futuristic. She starts looking down in partial shade. As wind moves the curtains, horizontal bands of golden sunlight travel slowly across her cheek, lips, and eyes. She raises her chin and turns from left profile into a clear three-quarter view, then gives a subtle thoughtful smile. The camera makes a slow push-in at eye level. Elegant natural portrait lighting, warm highlights, visible head motion, clean subject-background separation, no hands near face, no text, no logos. Audio: curtain fabric moving, distant city ambience, soft breath.

Prompt: A bright 65mm cockpit close-up frames a young seaplane pilot flying above a turquoise tropical archipelago in late morning. Sunlight fills the cream-colored cabin and clearly lights her face, amber aviator glasses, linen shirt, and weathered leather headset. She glances from the instruments to the horizon, smiles slightly, and banks the aircraft with one smooth movement. Through the softly blurred side windows, white beaches, coral lagoons, and small green islands drift below in vivid blue and jade layers; a few highlights move across brushed aluminum controls in the foreground. The camera remains close at passenger-seat height, with strong subject separation, natural reflections, shallow depth, bright airy color, and a relaxed adventure-film atmosphere. Audio: steady propeller, gentle cabin vibration, headset static, and wind around the floats.

(c) LTX 2.3

Figure 11 | Additional text-to-video comparisons of dense attention and Sol-Attn across different models.

Edit prompt: Turn the porcelain fox into transparent hand-blown glass containing a miniature thunderstorm with tiny clouds and lightning, while preserving its exact motion, silhouette, gallery, plinth, lighting, and camera path.

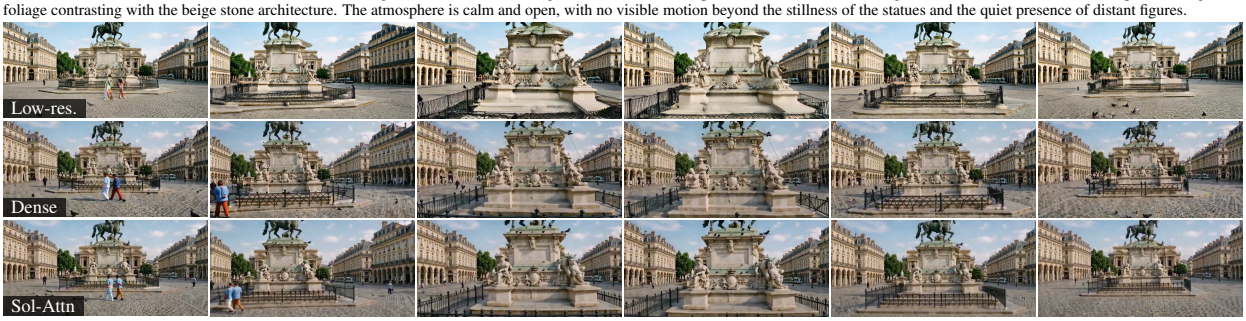


Edit prompt: Turn the red raincoat into a living garment of emerald moss and small white flowers, and make the reflecting pool show a star-filled night sky, while preserving the woman, walk cycle, architecture, framing, and daylight.

(a) Bernini

Prompt: A first-person view from a strictly stationary observation point across an immense dry lakebed bordered by low mountain ranges. A black sports car occupies the central foreground on the pale, compacted surface, aligned toward the open horizon beneath a vast blue sky. The environment is broad and minimal, with flat beige desert crust, faint tire-worn texture, distant rocky ridgelines, and a few thin clouds stretching across the upper sky. Bright midday sunlight creates crisp shadows under the vehicle and a clean, high-visibility atmosphere of speed, openness, and isolation. The observer's perspective remains fixed, with no dynamic camera movement and no actions taken by the person recording. Autonomous motion belongs to the world itself: dust trails sweep low across the ground, heat haze shimmers near the horizon, clouds drift slowly, and the car's tires kick up fine desert grit.

Prompt: A first-person view of a vast cobblestone plaza centered on an ornate equestrian statue atop a carved stone pedestal, flanked by symmetrical classical buildings with arched ground floors and uniform windows under slate roofs. The sky is pale blue with scattered clouds, casting soft daylight across the scene. Pigeons rest on the pavement in the foreground, while two pedestrians stroll near the statue's iron fence. Behind the monument, a grand neoclassical building with columns and sculptural reliefs anchors the background. Trees line the sides of the square, their green foliage contrasting with the beige stone architecture. The atmosphere is calm and open, with no visible motion beyond the stillness of the statues and the quiet presence of distant figures.



(b) SANA-WM

Figure 12 | Additional conditional-generation comparisons on Bernini and SANA-WM, with two cases per subfigure. Bernini rows show source, dense, and Sol-Attn-edited videos; SANA-WM rows show the low-resolution stage-1 video and both refined outputs.

References

[1] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

[2] Team Wan, Ang Wang, Baole Ai, Bin Wen, Chaojie Mao, Chen-Wei Xie, Di Chen, Feiwu Yu, Haiming Zhao, Jianxiao Yang, et al. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025.

[3] Weijie Kong, Qi Tian, Zijian Zhang, Rox Min, Zuozhuo Dai, Jin Zhou, Jiangfeng Xiong, Xin Li, Bo Wu, Jianwei Zhang, et al. HunyuanVideo: A systematic framework for large video generative models. *arXiv preprint arXiv:2412.03603*, 2024.

[4] Yoav HaCohen, Nisan Chiprut, Benny Brazowski, Daniel Shalem, Dudu Moshe, Eitan Richardson, Eran Levin, Guy Shiran, Nir Zabari, Ori Gordon, Poriya Panet, Sapir Weissbuch, Victor Kulikov, Yaki Bitterman, Zeev Melumian, and Ofir Bibi. LTX-Video: Realtime video latent diffusion. *arXiv preprint arXiv:2501.00103*, 2024.

[5] Lightricks. LTX-2.3 Model Card. https://huggingface.co/Lightricks/LTX-2.3, 2026. Model checkpoint family including ltx-2.3-22b-dev and distilled variants.

[6] Lichen Bai, Tianhao Zhang, Shitong Shao, Dingwei Tan, Qiyu Zhong, Zhengpeng Xie, Haopeng Li, Qinghao Huang, Dandan Shen, Tengjiao Ji, et al. MaineCoon: Pursuing a real-time audio-visual social world model. *arXiv preprint arXiv:2606.17800*, 2026.

[7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[8] Shuo Yang, Haocheng Xi, Yilong Zhao, Muyang Li, Jintao Zhang, Han Cai, Yujun Lin, Xiuyu Li, Chenfeng Xu, Kelly Peng, Jianfei Chen, Song Han, Kurt Keutzer, and Ion Stoica. Sparse VideoGen2: Accelerate video generation with sparse attention via semantic-aware permutation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[9] Jintao Zhang, Chendong Xiang, Haofeng Huang, Jia Wei, Haocheng Xi, Jun Zhu, and Jianfei Chen. SpargeAttention: Accurate and training-free sparse attention accelerating any model inference. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.

[10] Ruyi Xu, Guangxuan Xiao, Haofeng Huang, Junxian Guo, and Song Han. XAttention: Block sparse attention with antidiagonal scoring. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.

[11] Haopeng Li, Shitong Shao, Wenliang Zhong, Zikai Zhou, Lichen Bai, Hui Xiong, and Zeke Xie. PISA: Piecewise sparse attention is wiser for efficient diffusion transformers. *arXiv preprint arXiv:2602.01077*, 2026.

[12] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-Awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[13] Shitong Shao, Zikai Zhou, Haopeng Li, Yingwei Song, Wenliang Zhong, Lichen Bai, and Zeke Xie. LIVEditor-14B: Lightning unified video editing via in-context sparse attention. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026.

[14] Huiqiang Jiang, Yucheng Li, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir H. Abdi, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. MInference 1.0: Accelerating pre-filling for long-context LLMs via dynamic sparse attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[15] Guangxuan Xiao, Junxian Guo, Kasra Mazaheri, and Song Han. Optimizing mixture of block attention. *arXiv preprint arXiv:2511.11571*, 2025.

[16] Chenchen Liu, Junyi Chen, Lei Li, Lu Chi, Mingzhen Sun, Zhuoying Li, Yi Fu, Ruoyu Guo, Yiheng Wu, Ge Bai, and Zehuan Yuan. Bernini: Latent semantic planning for video diffusion. *arXiv preprint arXiv:2605.22344*, 2026.

[17] Haoyi Zhu, Haozhe Liu, Yuyang Zhao, Tian Ye, Junsong Chen, Jincheng Yu, Tong He, Song Han, and Enze Xie. SANA-WM: Efficient minute-scale world modeling with hybrid linear diffusion transformer. *arXiv preprint arXiv:2605.15178*, 2026.

[18] Ideogram AI. Ideogram 4. https://ideogram.ai/blog/ideogram-4.0/, 2026.

[19] Ziqi Huang, Yinan He, Jiashuo Yu, Fan Zhang, Chenyang Si, Yuming Jiang, Yuanhan Zhang, Tianxing Wu, Qingyang Jin, Nattapol Chanpaisit, Yaohui Wang, Xinyuan Chen, Limin Wang, Dahua Lin, Yu Qiao, and Ziwei Liu. VBench: Comprehensive benchmark suite for video generative models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[20] Niantong Li, Guangzheng Hu, Weixu Qiao, Ying Ba, Qichen Hong, et al. Qwen-Image-Bench: From generation to creation in text-to-image evaluation. *arXiv preprint arXiv:2605.28091*, 2026.

[21] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[22] Ted Zadouri, Markus Hoehnerbach, Jay Shah, Timmy Liu, Vijay Thakkar, and Tri Dao. FlashAttention-4: Algorithm and kernel pipelining co-design for asymmetric hardware scaling. *arXiv preprint arXiv:2603.05451*, 2026.

[23] Xin Zhou, Dingkang Liang, Kaijin Chen, Tianrui Feng, Xiwu Chen, Hongkai Lin, Yikang Ding, Feiyang Tan, Hengshuang Zhao, and Xiang Bai. Less is enough: Training-free video diffusion acceleration via runtime-adaptive caching. *arXiv preprint arXiv:2507.02860*, 2025.

[24] Feng Liu, Shiwei Zhang, Xiaofeng Wang, Yujie Wei, Haonan Qiu, Yuzhong Zhao, Yingya Zhang, Qixiang Ye, and Fang Wan. Timestep embedding tells: It's time to cache for video diffusion model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

[25] Jiacheng Liu, Chang Zou, Yuanhuiyi Lyu, Junjie Chen, and Linfeng Zhang. From reusing to forecasting: Accelerating diffusion models with TaylorSeers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025.

[26] Yitong Li, Junsong Chen, Haopeng Li, Haozhe Liu, Jincheng Yu, Ligeng Zhu, Ping Luo, Song Han, and Enze Xie. Sol video inference engine: Agent-native full-stack acceleration framework for efficient video generation. *arXiv preprint arXiv:2606.23743*, 2026.

[27] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.

[28] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

[29] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[30] Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *International Conference on Learning Representations (ICLR)*, 2020.

[31] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. *Transactions of the Association for Computational Linguistics*, 9:53–68, 2021.

[32] Yizhao Gao, Zhichen Zeng, Dayou Du, Shijie Cao, Peiyuan Zhou, Jiaxing Qi, Junjie Lai, Hayden So, Ting Cao, Fan Yang, and Mao Yang. SeerAttention: Self-distilled attention gating for efficient long-context prefilling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[33] Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Yuxing Wei, Lean Wang, Zhiping Xiao, Yuqing Wang, Chong Ruan, Ming Zhang, Wenfeng Liang, and Wangding Zeng. Native sparse attention: Hardware-aligned and natively trainable sparse attention. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers*, 2025.

[34] Enzhe Lu, Zhejun Jiang, Jingyuan Liu, Yulun Du, Tao Jiang, Chao Hong, Shaowei Liu, Weiran He, Enming Yuan, Yuzhi Wang, Zhiqi Huang, Huan Yuan, Suting Xu, Xinran Xu, Guokun Lai, Yanru Chen, Huabin Zheng, Junjie Yan, Jianlin Su, Yuxin Wu, Yutao Zhang, Zhilin Yang, Xinyu Zhou, Mingxing Zhang, and Jiezhong Qiu. MoBA: Mixture of block attention for long-context LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[35] Haocheng Xi, Shuo Yang, Yilong Zhao, Chenfeng Xu, Muyang Li, Xiuyu Li, Yujun Lin, Han Cai, Jintao Zhang, Dacheng Li, Jianfei Chen, Ion Stoica, Kurt Keutzer, and Song Han. Sparse video-gen: Accelerating video diffusion transformers with spatial-temporal sparsity. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.

[36] Xingyang Li, Muyang Li, Tianle Cai, Haocheng Xi, Shuo Yang, Yujun Lin, Lvmin Zhang, Songlin Yang, Jinbo Hu, Kelly Peng, Maneesh Agrawala, Ion Stoica, Kurt Keutzer, and Song Han. Radial attention: $\mathcal{O}(n \log n)$ sparse attention with energy decay for long video generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[37] Peiyuan Zhang, Yongqi Chen, Runlong Su, Hangliang Ding, Ion Stoica, Zhengzhong Liu, and Hao Zhang. Fast video generation with sliding tile attention. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.

[38] Pengtao Chen, Xianfang Zeng, Maosen Zhao, Mingzhu Shen, Wei Cheng, Gang Yu, and Tao Chen. Sparse-vDiT: Unleashing the power of sparse attention to accelerate video diffusion transformers. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2026.

[39] Peiyuan Zhang, Yongqi Chen, Haofeng Huang, Will Lin, Zhengzhong Liu, Ion Stoica, Eric P. Xing, and Hao Zhang. Faster video diffusion with trainable sparse attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[40] Jie Hu, Zixiang Gao, Yutong He, and Kun Yuan. DFSAttn: Dynamic fine-grained sparse attention for efficient video generation. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026.

[41] Akide Liu, Zeyu Zhang, Zhexin Li, Xuehai Bai, Yuanjie Xing, Yizeng Han, Jiasheng Tang, Jichao Wu, Mingyang Yang, Weihua Chen, Jiahao He, Yuanyu He, Fan Wang, Reza Haffari, and Bohan Zhuang. FPSAttention: Training-aware FP8 and sparsity co-design for fast video diffusion. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[42] Jintao Zhang, Kai Jiang, Chendong Xiang, Weiqi Feng, Yuezhou Hu, Haocheng Xi, Jianfei Chen, and Jun Zhu. SpargeAttention2: Trainable sparse attention via hybrid top-k+top-p masking and distillation fine-tuning. *arXiv preprint arXiv:2602.13515*, 2026.

[43] Chaofan Lin, Jiaming Tang, Shuo Yang, Hanshuo Wang, Tian Tang, Boyu Tian, Ion Stoica, Song Han, and Mingyu Gao. Twilight: Adaptive attention sparsity with hierarchical top-p pruning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[44] Anmin Liu, Ruixuan Yang, Huiqiang Jiang, Bin Lin, Minmin Sun, Yong Li, Chen Zhang, and Tao Xie. VecAttention: Vector-wise sparse attention for accelerating long context inference. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026.

[45] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. QUEST: Query-aware sparsity for efficient long-context LLM inference. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

[46] Heejun Lee, Geon Park, Youngwan Lee, Jaduk Suh, Jina Kim, Wonyoung Jeong, Bumsik Kim, Hyemin Lee, Myeongjae Jeon, and Sung Ju Hwang. A training-free sub-quadratic cost transformer model serving framework with hierarchically pruned attention. In *International Conference on Learning Representations (ICLR)*, 2025.

[47] Xunhao Lai, Jianqiao Lu, Yao Luo, Yiyuan Ma, and Xun Zhou. FlexPrefill: A context-aware sparse attention mechanism for efficient long-sequence inference. In *International Conference on Learning Representations (ICLR)*, 2025.

[48] Aditya Desai, Shuo Yang, Alejandro Cuadron, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. HashAttention: Semantic sparsity for faster inference. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.

[49] Xuanyi Zhou, Qiuyang Mang, Shuo Yang, Haocheng Xi, Jintao Zhang, Huanzhi Mao, Joseph E. Gonzalez, Kurt Keutzer, Ion Stoica, and Alvin Cheung. SVG-EAR: Parameter-free linear compensation for sparse video generation via error-aware routing. *arXiv preprint arXiv:2603.08982*, 2026.

[50] Wenhu Zhang, Yiming Wu, Huanyu Wang, Yaoyang Liu, Huanzhang Dou, Senqiao Yang, Sitong Wu, Hanbin Zhao, and Jiaya Jia. Efficient long-context modeling in diffusion language models via block approximate sparse attention. *arXiv preprint arXiv:2605.19726*, 2026.

[51] Jintao Zhang, Haoxu Wang, Kai Jiang, Shuo Yang, Kaiwen Zheng, Haocheng Xi, Ziteng Wang, Hongzhou Zhu, Min Zhao, Ion Stoica, Joseph E. Gonzalez, Jianfei Chen, and Jun Zhu. SLA: Beyond sparsity in diffusion transformers via fine-tunable sparse-linear attention. In *International Conference on Learning Representations (ICLR)*, 2026.

[52] Jintao Zhang, Haoxu Wang, Kai Jiang, Kaiwen Zheng, Youhe Jiang, Ion Stoica, Jianfei Chen, Jun Zhu, and Joseph E. Gonzalez. SLA2: Sparse-linear attention with learnable routing and QAT. *arXiv preprint arXiv:2602.12675*, 2026.